

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Andriy Shevtiyan - 5024231080

2026

1 Pendahuluan

1.1 Latar Belakang

Hampir setiap perangkat elektronik modern, mulai dari mesin cuci, termostat pintar, dan node sensor IoT hingga lengan robot di lini produksi, dikendalikan oleh sebuah mikrokontroler. Mikrokontroler merupakan sistem komputer berukuran kecil yang dikemas dalam satu keping chip; di dalamnya sudah terdapat prosesor, memori, serta berbagai periferal seperti GPIO, ADC, timer, dan antarmuka komunikasi serial. Dengan perpaduan tersebut, mikrokontroler mampu menerima informasi dari lingkungan sekitarnya sekaligus mengendalikan perangkat lain secara langsung. Oleh sebab itu, kemampuan memprogram mikrokontroler, menentukan platform yang sesuai dengan kebutuhan, serta menghubungkannya dengan komponen fisik merupakan kompetensi dasar yang perlu dikuasai oleh mahasiswa Teknik Komputer.

Pada Modul 1, praktikan dikenalkan dengan dua board yang akan digunakan selama rangkaian praktikum, yakni STM32 Blackpill yang diprogram melalui framework Arduino, serta ESP32 yang dapat diprogram dengan framework ESP-IDF maupun Arduino. Selain itu, modul ini membahas tiga keterampilan dasar yang hampir selalu dibutuhkan dalam perancangan sistem tertanam, yaitu pembacaan input digital dari tombol dengan konfigurasi pull-up dan pull-down, penanganan sinyal tidak stabil akibat pantulan kontak mekanik (debouncing), serta pembacaan tegangan analog menggunakan ADC. Inti permasalahan yang dibahas adalah bagaimana memastikan masukan dari dunia nyata terbaca dengan benar oleh mikrokontroler: nilainya tidak acak karena pin dibiarkan mengambang (floating), satu kali penekanan tidak terhitung berkali-kali (bouncing), dan besaran analog dikonversi dengan resolusi yang memadai. Di samping itu, praktikan juga perlu menyiapkan lingkungan pengembangan agar kode dapat ditulis, dikompilasi, dan diunggah ke board. Bekal ini menjadi dasar bagi modul-modul selanjutnya, antara lain sensor dan aktuator, komunikasi serial, interrupt dan multitasking, hingga sistem kendali.

1.2 Dasar Teori

1.2.1 PlatformIO dan VSCode

PlatformIO merupakan lingkungan pengembangan perangkat lunak tertanam yang bersifat lintas platform dan dipasang sebagai ekstensi pada Visual Studio Code. Melalui satu antarmuka yang sama, PlatformIO dapat menangani beragam board dan framework, misalnya Arduino, STM32Cube/HAL, dan ESP-IDF. Setiap project PlatformIO memiliki susunan folder yang baku: berkas `platformio.ini` menyimpan pengaturan board, framework, dan metode upload; folder `src/` berisi kode program utama; `include/` menampung berkas header; dan `lib/` digunakan untuk pustaka tambahan. Karena seluruh konfigurasi terpusat di `platformio.ini`, chip yang sama dapat diprogram menggunakan framework lain cukup dengan menyunting berkas tersebut.

1.2.2 STM32 Blackpill, ST-Link, dan Framework Arduino

STM32 Blackpill merupakan board pengembangan berbasis mikrokontroler STM32F401CC. Chip ini berinti ARM Cortex-M4 yang dilengkapi unit floating-point (FPU), dapat bekerja hingga frekuensi 84 MHz, serta memiliki memori Flash 256 KB dan RAM 64 KB. Pada board ini, LED bawaan terhubung ke pin PC13 dengan logika aktif LOW (menyala ketika pin bernilai 0), sedangkan tombol pengguna terhubung ke pin PA0. Blackpill tidak dibekali programmer di atas board-nya, sehingga proses flashing pada praktikum ini dilakukan dengan ST-Link eksternal melalui antarmuka SWD yang memerlukan empat jalur, yaitu SWDIO, SWCLK, GND, dan 3,3 V.

Pemrogramannya memakai framework Arduino versi STM32duino yang menyediakan fungsi-fungsi seperti `pinMode()`, `digitalWrite()`, dan `delay()`; meskipun begitu, fungsi-fungsi tersebut pada dasarnya dibangun di atas pustaka HAL milik STMicroelectronics.

1.2.3 ESP32 dan Framework ESP-IDF

ESP32 merupakan mikrokontroler 32-bit berinti ganda (dual-core) yang telah mengintegrasikan konektivitas WiFi dan Bluetooth dalam satu chip. Board ESP32 DevKit sudah dilengkapi chip konverter USB-to-Serial, sehingga pemrogramannya cukup dilakukan dengan menghubungkan board ke komputer menggunakan kabel USB. ESP-IDF (Espressif IoT Development Framework) adalah framework resmi buatan Espressif yang berjalan di atas sistem operasi waktu nyata FreeRTOS. Program ESP-IDF dimulai dari fungsi `app_main()`, dan jeda waktu diatur menggunakan `vTaskDelay()`. Sementara itu, framework Arduino menggunakan pola `setup()` dan `loop()` serta fungsi `delay()`. Arduino menawarkan abstraksi yang lebih tinggi sehingga lebih praktis digunakan, tetapi akses ke fitur-fitur RTOS menjadi lebih terbatas dibandingkan dengan ESP-IDF.

1.2.4 Resistor Pull-up dan Pull-down

Apabila sebuah pin input tidak dihubungkan ke tegangan acuan mana pun, pin tersebut berada dalam kondisi mengambang (floating) sehingga nilai bacaannya mudah berubah-ubah akibat derau (noise). Untuk menghindarinya digunakan resistor pull-up atau pull-down. Pada rangkaian pull-up, resistor menghubungkan pin ke VCC dan tombol dipasang ke GND, sehingga pin bernilai HIGH dalam keadaan diam dan berubah menjadi LOW ketika tombol ditekan. Sebaliknya, pada rangkaian pull-down resistor menghubungkan pin ke GND dan tombol dipasang ke VCC, sehingga pin bernilai LOW saat diam dan HIGH saat tombol ditekan. Resistor tersebut dapat dipasang secara fisik (nilai yang lazim digunakan adalah 10 kΩ) ataupun diaktifkan secara internal melalui kode, misalnya dengan mode `INPUT_PULLUP` dan `INPUT_PULLDOWN` pada ESP32. Perlu diperhatikan bahwa pin GPIO 34–39 pada ESP32 hanya dapat berfungsi sebagai input dan tidak memiliki resistor pull internal.

1.2.5 Debouncing

Saat tombol mekanik ditekan atau dilepas, bilah kontak logam di dalamnya tidak langsung menempel atau terpisah dengan sempurna, melainkan bergetar dan memantul beberapa kali. Akibatnya, sinyal yang dihasilkan berganti-ganti antara HIGH dan LOW dalam rentang waktu yang sangat singkat; fenomena ini disebut bouncing. Jika tidak ditangani, mikrokontroler dapat membaca satu kali penekanan sebagai beberapa penekanan. Teknik debouncing bertujuan agar hanya satu perubahan keadaan yang dianggap valid, salah satunya dengan menunggu hingga sinyal tetap stabil selama sekitar 20–50 ms sebelum perubahan tersebut diterima.

1.2.6 ADC

Di samping berfungsi sebagai output dan input digital, pin GPIO tertentu juga dapat membaca sinyal analog melalui ADC (Analog-to-Digital Converter), yaitu rangkaian yang mengonversi tegangan yang bersifat kontinu menjadi angka digital. ADC internal pada ESP32 secara bawaan memiliki resolusi 12 bit, sehingga menghasilkan nilai 0 hingga 4095 untuk rentang tegangan sekitar 0–3,3 V. Pada framework Arduino, pembacaan ini dilakukan dengan fungsi `analogRead()`. Penggunaan kanal ADC1 (contohnya GPIO 32–39) lebih dianjurkan dibandingkan ADC2, sebab ADC2 tidak dapat digunakan secara leluasa ketika WiFi sedang aktif.

Materi dasar teori di atas bersumber dari Modul 1 Praktikum Sistem Tertanam.

2 Tugas Pendahuluan

Bagian ini memuat jawaban atas soal-soal tugas pendahuluan beserta uraiannya.

1. Jelaskan perbedaan mendasar antara framework Arduino dan ESP-IDF dalam memprogram mikrokontroler, termasuk kelebihan dan kekurangan masing-masing.

   Jawaban: Perbedaan utama keduanya terletak pada tingkat abstraksi dan kedekatannya dengan perangkat keras. Framework Arduino menyembunyikan banyak detail teknis: kode cukup ditulis dalam dua fungsi, yaitu `setup()` yang dieksekusi satu kali saat board menyala dan `loop()` yang dieksekusi terus-menerus, dengan bantuan fungsi sederhana seperti `pinMode()`, `digitalWrite()`, dan `delay()`. ESP-IDF, di sisi lain, merupakan framework bawaan Espressif yang berbasis FreeRTOS. Eksekusi program diawali dari `app_main()`, aplikasi disusun dalam bentuk task dan component, dan jeda waktu ditulis dengan `vTaskDelay(pdMS_TO_TICKS(ms))`.

   - **Kelebihan Arduino:** sintaksnya sederhana sehingga ramah bagi pemula, kode yang ditulis lebih pendek, mudah dipindahkan ke board lain, dan didukung oleh banyak pustaka dari komunitas.
   - **Kekurangan Arduino:** karena detail perangkat keras dan RTOS disembunyikan, pengembang memiliki kendali yang terbatas untuk mengatur dan mengoptimalkan sistem pada level rendah.
   - **Kelebihan ESP-IDF:** memberikan akses menyeluruh ke kemampuan ESP32, seperti pengaturan WiFi tingkat rendah serta fitur FreeRTOS (task, semaphore, dan queue), sehingga sesuai untuk membangun produk IoT yang kompleks.
   - **Kekurangan ESP-IDF:** lebih sulit dipelajari, kode yang dibutuhkan lebih panjang, dan hanya dapat dipakai pada chip keluaran Espressif.

2. Mengapa board STM32 Blackpill memerlukan programmer eksternal (ST-Link) untuk di-flash, sedangkan ESP32 dapat diprogram langsung melalui kabel USB? Kaitkan jawaban Anda dengan keberadaan chip USB-to-Serial.

   Jawaban: Perbedaannya terletak pada ada atau tidaknya penghubung antara port USB komputer dan mikrokontroler. Board ESP32 DevKit telah dilengkapi chip USB-to-Serial, contohnya CP2102 atau CH340, yang mengonversi data USB dari komputer menjadi sinyal UART yang dapat diterima ESP32. Dengan begitu, komputer mengenali board sebagai port serial (COM) dan firmware dapat diunggah langsung lewat kabel USB. Sebaliknya, STM32 Blackpill tidak memiliki chip USB-to-Serial ataupun programmer bawaan pada board-nya. Oleh karena itu, firmware diunggah melalui antarmuka debug SWD (SWDIO, SWCLK, GND, dan 3,3 V) dengan bantuan perangkat terpisah, yaitu ST-Link, yang berperan sebagai penghubung antara komputer dan chip STM32.

   Catatan: penjelasan ini mengacu pada metode yang digunakan dalam praktikum. Sebenarnya STM32F401 juga memiliki bootloader internal yang memungkinkan pemrograman melalui USB dalam mode DFU, tetapi mode tersebut tidak digunakan pada praktikum ini.

3. Jelaskan perbedaan kondisi logika (HIGH/LOW) pada konfigurasi pull-up dan pull-down resistor, baik saat tombol tidak ditekan maupun saat ditekan.

   Jawaban: Ringkasan kondisi logika pada kedua konfigurasi ditunjukkan pada Tabel 1.

Tabel 1: Kondisi logika pin pada konfigurasi pull-up dan pull-down

Konfigurasi	Tombol dilepas	Tombol ditekan
Pull-up (resistor ke VCC, tombol ke GND)	HIGH	LOW
Pull-down (resistor ke GND, tombol ke VCC)	LOW	HIGH

Pada konfigurasi pull-up, resistor menahan pin pada tegangan VCC selama tombol dilepas sehingga pin bernilai HIGH; begitu tombol ditekan, pin tersambung langsung ke GND dan nilainya turun menjadi LOW. Konfigurasi pull-down bekerja dengan logika sebaliknya: resistor menahan pin pada GND sehingga bernilai LOW saat tombol dilepas, lalu pin bernilai HIGH ketika tombol ditekan karena tersambung ke VCC. Dengan kata lain, pull-up menghasilkan logika aktif LOW, sedangkan pull-down menghasilkan logika aktif HIGH, dan keduanya sama-sama mencegah pin berada dalam kondisi floating.

4. Jelaskan fenomena bouncing pada tombol mekanik — mengapa hal ini bisa terjadi — dan sebutkan minimal dua pendekatan untuk mengatasinya.

   Jawaban: Bouncing muncul karena kontak di dalam tombol terbuat dari logam yang bersifat elastis. Ketika kedua kontak saling bertemu atau berpisah, kontak tersebut tidak langsung diam, tetapi memantul beberapa kali dalam hitungan milidetik. Selama proses itu sinyal berubah-ubah antara HIGH dan LOW sebelum akhirnya stabil, dan karena mikrokontroler membaca pin dengan sangat cepat, setiap perubahan tersebut dapat dianggap sebagai penekanan yang terpisah. Beberapa cara untuk mengatasinya adalah sebagai berikut.

   - **Debouncing berbasis perangkat lunak:** perubahan keadaan tombol baru diterima apabila keadaan yang baru bertahan tanpa berubah selama selang waktu tertentu, misalnya 50 ms, dengan memanfaatkan fungsi `millis()` untuk mencatat waktu perubahan terakhir.
   - **Debouncing berbasis perangkat keras:** menambahkan rangkaian filter RC, yaitu resistor bersama kapasitor yang dipasang paralel dengan tombol, dan dapat dikombinasikan dengan Schmitt trigger agar transisi yang cepat dan tidak bersih teredam sebelum mencapai pin mikrokontroler.

5. Bagaimana ADC (*Analog-to-Digital Converter*) mengubah sinyal analog menjadi nilai digital? Jelaskan pula pengaruh resolusi bit ADC terhadap presisi hasil pembacaan.

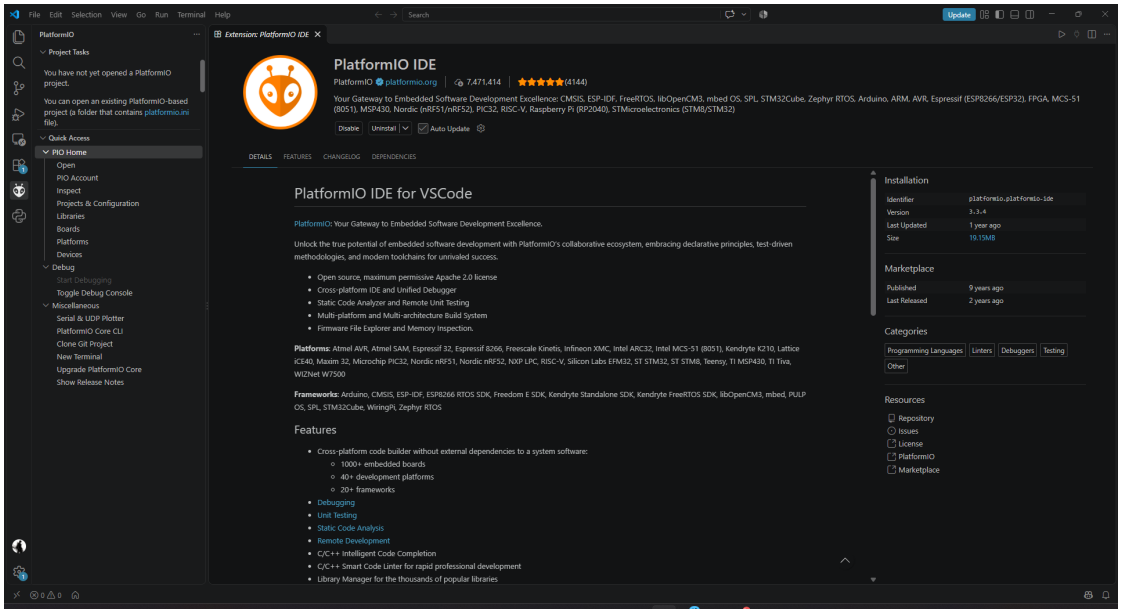
   Jawaban: Proses konversi pada ADC berlangsung dalam tiga langkah. Pertama, *sampling*, yaitu pengambilan nilai tegangan sinyal pada interval waktu tertentu. Kedua, *kuantisasi*, yaitu pembulatan nilai tegangan tersebut ke salah satu dari 2^N tingkat diskrit yang tersedia. Ketiga, *pengkodean*, yaitu penyajian tingkat hasil kuantisasi dalam bentuk bilangan biner. Pada ESP32 yang bekerja dengan resolusi 12 bit dan rentang 0–3,3 V, keluaran ADC berada pada kisaran 0 hingga 4095. Secara umum, ADC N-bit menyediakan 2^N tingkat, sehingga perubahan tegangan terkecil yang masih dapat dideteksi (1 LSB) adalah $V_{ref}/2^N$:

   - 8-bit: $3{,}3/256 \approx 12{,}9$ mV per tingkat
   - 10-bit: $3{,}3/1024 \approx 3{,}2$ mV per tingkat
   - 12-bit: $3{,}3/4096 \approx 0{,}81$ mV per tingkat

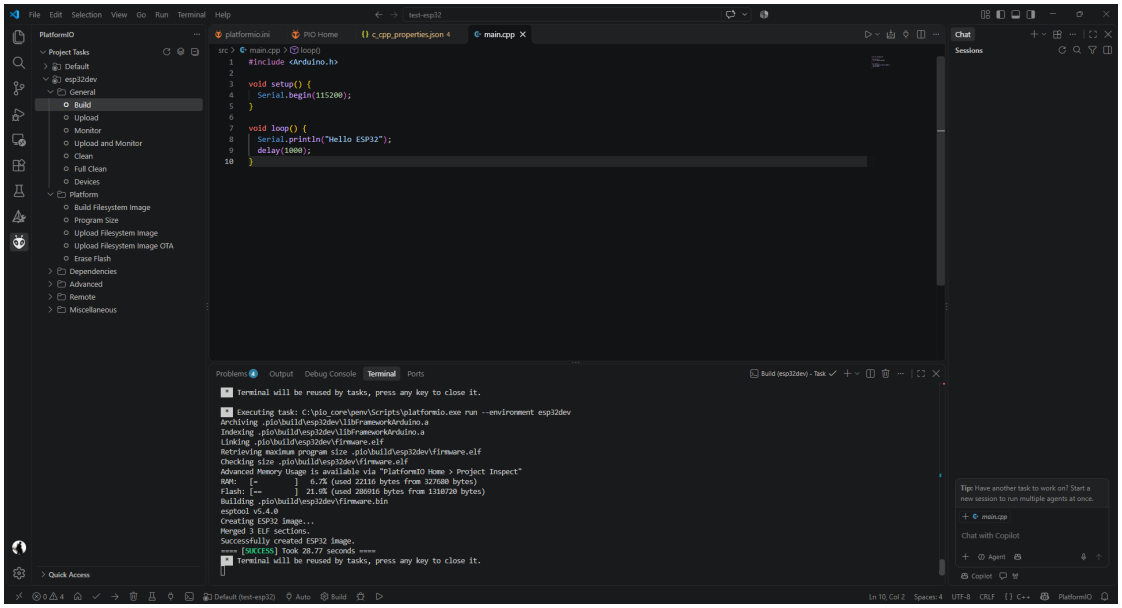
   Dari perhitungan tersebut terlihat bahwa makin tinggi resolusi bit ADC, makin kecil selisih tegangan yang dapat dibedakan, sehingga hasil pembacaan menjadi lebih presisi.

6. Lakukan set-up untuk tools: VSCode, ekstensi PlatformIO (dengan framework ESP-IDF dan STM32), dan ekstensi Serial Plotter pada laptop kalian, berikan screenshot bahwa laptop kalian sudah ter-set-up dengan baik.

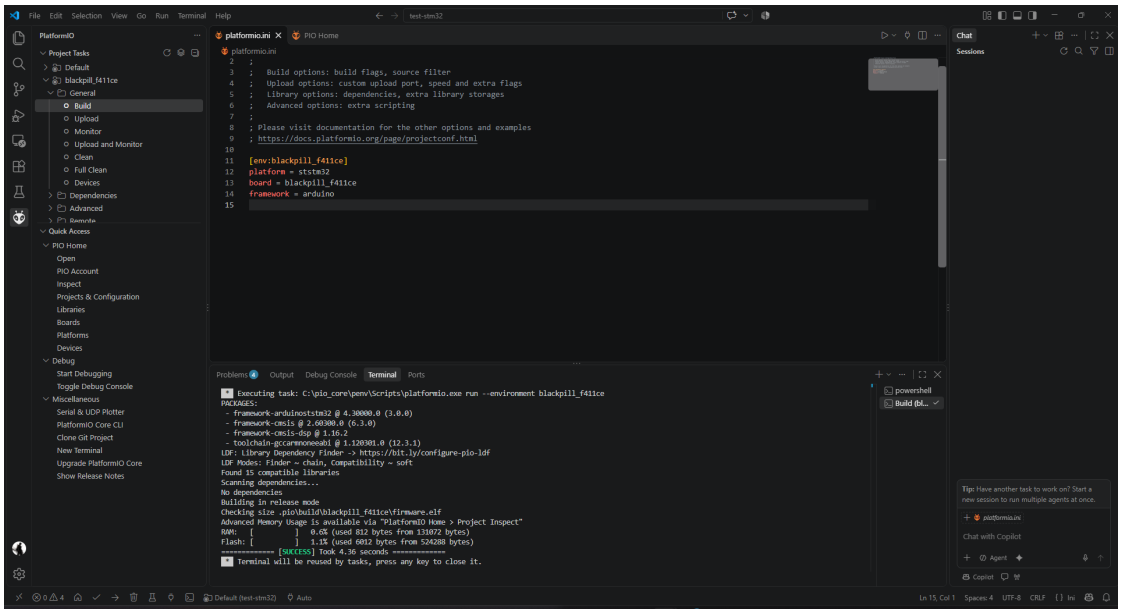
   Jawaban: Laptop telah disiapkan dengan memasang Visual Studio Code, ekstensi PlatformIO IDE beserta platform Espressif 32 dan ST STM32, serta ekstensi Serial Plotter. Ekstensi PlatformIO IDE yang sudah terpasang ditunjukkan pada Gambar 1. Untuk memastikan kedua platform dapat digunakan, dibuat project uji untuk ESP32 dan STM32 Blackpill yang keduanya berhasil di-build tanpa error (Gambar 2 dan Gambar 3). Adapun ekstensi Serial Plotter yang sudah terpasang ditunjukkan pada Gambar 4.



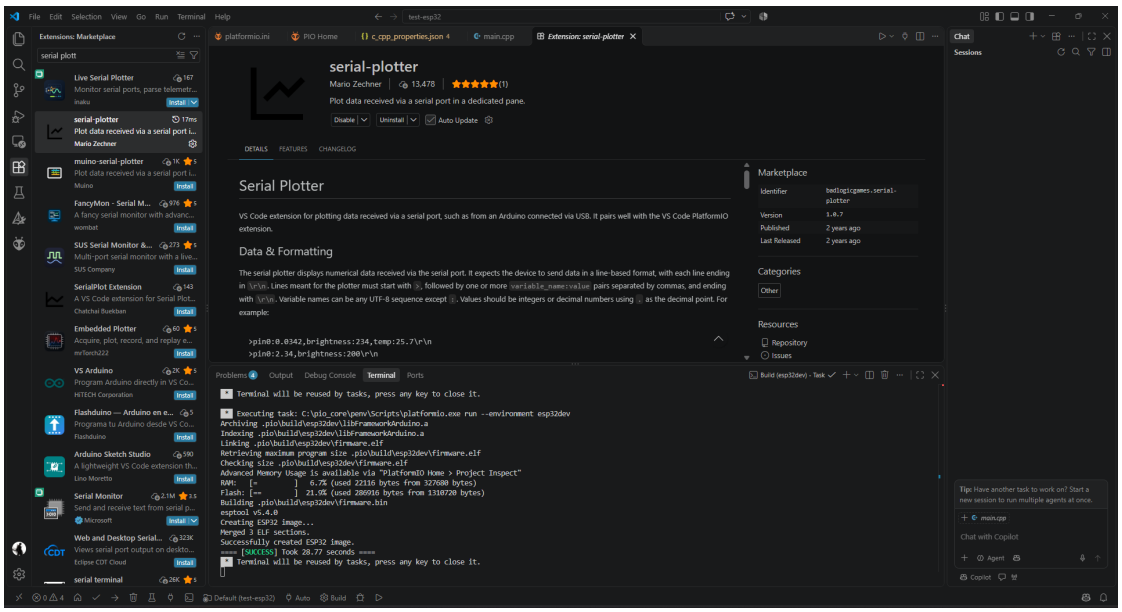
Gambar 1: Ekstensi PlatformIO IDE telah terpasang di Visual Studio Code



Gambar 2: Project uji ESP32 berhasil di-build dengan status SUCCESS



Gambar 3: Konfigurasi `platformio.ini` untuk STM32 Blackpill beserta hasil build berstatus SUCCESS



Gambar 4: Ekstensi Serial Plotter telah terpasang di Visual Studio Code